Analyse du livrable P1

Référentiel de travail construit ensemble — projet ECE, module 1.

1. Le but du projet

Traiter un grand volume de logs pour obtenir des informations sur l'activité et la sécurité d'un système. Un log est la trace d'un événement : connexion réussie, échec de connexion, requête ou erreur de service.

P1 lit, parse et valide les événements. P2 les nettoie, les normalise et applique les règles de sécurité. P3 calcule les statistiques et détecte les attaques. Le programme affiche ensuite le bilan.

Ton rôle peut se formuler ainsi : P1 lit les logs progressivement, transforme chaque ligne valide en un dictionnaire conforme au contrat et permet à P2 de les traiter sans charger tout le fichier en mémoire.

2. Un log dans notre programme

L'équipe a choisi JSON Lines (JSONL) : chaque ligne contient un objet JSON représentant un événement, sans tableau englobant.

Exemple simplifié :

```
{"level":"ERROR","user_id":"alice"}
```

Avant parsing, la ligne est une chaîne Python (str) contenant du texte au format JSON. Après parsing, l'événement est un dictionnaire (dict). Une ligne valide correspond à un événement, donc à un dictionnaire.

Pour 50 millions de logs valides, P1 fournit 50 millions de dictionnaires successivement. Il ne les conserve pas tous ensemble.

3. Pourquoi un dictionnaire ?

La lecture récupère le texte. Le parsing interprète sa structure pour accéder aux informations par leur nom.

```
evenement["level"]    # "ERROR"
evenement["user_id"]  # "alice"
```

P2 et P3 travaillent ainsi sur des champs identifiés au lieu de rechercher les informations dans du texte. Le dictionnaire organise les données ; il ne réduit pas à lui seul leur consommation mémoire.

4. La contrainte mémoire

Les chaînes, listes et dictionnaires sont stockés en RAM. Conserver tous les événements dans une liste peut saturer cette mémoire.

Le traitement progressif suit ce chemin : lire une ligne, la transformer en

dictionnaire, traiter cet événement dans la chaîne, puis passer à la suivante. Le fichier original reste sur disque. Les objets nécessaires au traitement courant sont en RAM.

Quand plus aucun module ne conserve de référence à un événement, Python peut libérer sa mémoire. Pour P1, la mémoire dépend surtout de la plus grande ligne et des tampons de lecture. Une ligne gigantesque reste donc une limite.

P3 ne stocke pas tous les événements pour les compter :

```
Départ                  : erreurs = 0
Premier événement ERROR  : erreurs = 1
Deuxième événement ERROR : erreurs = 2
```

Un compteur atteignant plusieurs millions prend beaucoup moins de mémoire que les millions de dictionnaires correspondants. P3 conserve aussi des IP distinctes et des états de détection : ces structures peuvent croître avec le nombre d'IP ou d'utilisateurs différents.

5. Lire un fichier : read_lines()

- Entrée : le chemin du fichier.
- Travail : ouvrir le fichier et parcourir ses lignes.
- Sortie progressive : une ligne de texte à chaque demande.
- Fin : fermer le fichier.
- Hors de son rôle : parsing et règles de sécurité.

Notions associées :

- def : regrouper le comportement dans une fonction.
- Paramètre : recevoir le chemin pour réutiliser la fonction.
- open() : ouvrir le fichier en lecture.
- for : parcourir les lignes successivement.
- yield : fournir une ligne et suspendre la fonction.
- with open(...) : organiser la fermeture du fichier, même en cas d'erreur.

Avec un générateur suspendu sur yield, le fichier reste ouvert. Il est fermé à l'épuisement ou à la fermeture explicite du générateur. Si un consommateur arrête le parcours tôt, il doit fermer le générateur.

6. yield, générateurs et lecture paresseuse

Une fonction contenant yield est une fonction génératrice. Son appel crée un générateur ; son corps commence à s'exécuter lorsqu'on lui demande une valeur.

yield fournit une valeur, suspend la fonction et permet de reprendre à la demande suivante. Il ne surveille pas la fin du traitement d'un événement : c'est la prochaine demande qui provoque la reprise.

```
P2 demande une ligne
P1 avance jusqu'à yield et fournit la ligne
P1 reste suspendu pendant le traitement suivant
P2 demande la prochaine ligne
P1 reprend après yield
```

Une lecture paresseuse produit les valeurs à mesure qu'elles sont demandées. Un générateur épuisé ne recommence pas : pour relire, on crée un nouveau lecteur.

Iterator[str] signifie : un itérateur qui fournit des chaînes de caractères. C'est une annotation de contrat, pas une liste ni une validation automatique des types.

Différence importante : yield fournit une valeur et suspend ; return termine l'appel d'une fonction. Le lecteur utilise yield pour plusieurs lignes. Le parseur utilise return parce qu'il traite une seule ligne par appel.

7. Lire plusieurs fichiers : read_sources()

Cette fonction reçoit plusieurs chemins et réutilise read_lines() pour chaque fichier, dans l'ordre fourni. Elle lit A entièrement, puis B. Aucun tri global n'est prévu.

Sa sortie reste une ligne de texte à la fois. Le parseur n'a pas besoin de savoir si le flux vient d'un ou de plusieurs fichiers.

- for parcourt les chemins.
- read_lines() fournit les lignes du fichier courant.
- yield from transmet les valeurs de cet autre générateur.

Un fichier vide ne fournit aucune valeur : le parseur n'est pas appelé et le lecteur multisource passe au fichier suivant. Si A est vide et B contient trois événements valides, trois événements sont traités.

Attention : notre fonction read_lines() est différente de la méthode readlines(), qui rassemble toutes les lignes dans une liste et ne convient pas au chargement d'un fichier de 10 Go.

8. Fournir les chemins dans la commande

Décision prise : les chemins ne sont pas écrits directement dans main.py. Ils sont fournis par l'utilisateur au lancement :

```
.\.venv\Scripts\python.exe main.py A.jsonl B.jsonl
```

main.py récupère les chemins puis les transmet au lecteur multisource. Lire d'autres fichiers demande de changer la commande, pas le code du lecteur. Le mécanisme de récupération des arguments devra être expliqué avant utilisation.

9. Parser et valider : parse_line()

- Entrée : une chaîne contenant une ligne JSON.
- Si la ligne est conforme : retourner le dictionnaire de l'événement.
- Si la ligne est invalide : signaler la cause et retourner None.
- Si la ligne est vide : retourner None sans avertissement d'erreur.

None signifie : aucun événement valide à transmettre. On ne retourne pas un dictionnaire vide pour représenter un rejet.

Deux étapes sont distinctes : décoder le JSON, puis valider la structure obtenue. Du JSON valide peut représenter autre chose qu'un dictionnaire : il faut donc vérifier que la racine est bien un objet JSON devenu un dict.

```
{"level":       -> JSON incomplet, décodage impossible
```

```
{"level": ""}   -> JSON valide, chaîne vide à examiner ensuite
```

10. Le contrat des six champs

Ces champs sont obligatoires selon le guide d'équipe :

- timestamp : date et heure, converties en datetime.
- level : chaîne str, conservée avant normalisation P2.
- ip : chaîne str représentant l'IP source.
- user_id : chaîne str représentant l'utilisateur.
- connection : tuple (IP str, port int, protocole str).
- payload : contenu sous forme de dict ou str.

Présence des champs

```
"timestamp" in evenement
```

in teste la présence d'une clé et retourne True ou False. Présence et validité sont deux contrôles différents. Si un champ obligatoire manque, P1 rejette sans inventer sa valeur.

timestamp

La chaîne 2026-11-05T10:00:00 représente le 5 novembre 2026 à 10 h. Le T sépare date et heure.

P1 vérifie la présence du champ, son type str et la possibilité de le convertir en datetime. Avec "bonjour", le champ existe mais la conversion est impossible : l'événement est rejeté.

On vérifie que la date est exploitable, pas qu'elle correspond réellement à ce qui s'est passé. Aucun fuseau n'est inventé.

connection

```
["192.168.1.10", 443, "HTTPS"]  # liste issue du JSON
("192.168.1.10", 443, "HTTPS")  # tuple après P1
```

Un tuple est immuable : on peut lire ses éléments, mais pas les remplacer directement. P1 vérifie trois éléments, leurs types str/int/str, puis convertit avec tuple().

len() vérifie le nombre d'éléments ; isinstance() vérifie les types :

```
len(["192.168.1.10", 443, "HTTPS"])  # 3
isinstance(443, int)                # True
isinstance("443", int)              # False
isinstance("HTTPS", str)            # True
```

La chaîne "443" n'est pas un entier. La politique retenue exige un port déjà entier, sans conversion automatique. Un booléen est aussi rejeté comme port : bool hérite de int en Python, ce qui impose une attention particulière au contrôle.

Si le protocole manque, on rejette l'événement sans l'inventer.

payload

payload désigne le contenu de l'événement, pas son acceptation :

```
{"event": "login", "status": "success"}  # dict
"Service unavailable"                    # str
```

Les deux formes sont autorisées. La condition étudiée est :

```
isinstance(payload, dict) or isinstance(payload, str)
```

or signifie qu'au moins un test doit être vrai. Le nombre 42 ne respecte aucun des deux types : rejet. Un payload correct ne suffit pas si un autre champ obligatoire manque.

Frontière avec P2

P1 prépare les types ; P2 normalise notamment les niveaux et protocoles. P1 préserve donc " warning ", que P2 pourra transformer en "WARNING". Les niveaux inconnus sont filtrés par P2 après normalisation.

Les champs supplémentaires sont conservés. La validation métier des IP, les bornes des ports et les protocoles autorisés ne sont pas définis : ne pas inventer de restrictions.

11. Transmettre les données à P2

Le guide prévoit que build_pipeline() de P2 reçoive les lignes, appelle parse_line() de P1 puis écarte les retours None :

```
read_sources() fournit une ligne
P2 appelle parse_line(ligne)
Le parseur retourne dict ou None à P2
P2 traite le dictionnaire ou écarte None
La chaîne demande la ligne suivante
```

return None termine seulement l'appel du parseur. Il ne termine ni le lecteur ni le fichier entier. Le parseur peut être appelé de nouveau pour la prochaine ligne.

Iterator[dict] décrit le flux de dictionnaires valides, sans None transmis à la suite et sans liste globale. Ne pas parser une première fois dans main.py avant d'envoyer les données à build_pipeline : ce dernier doit recevoir les lignes.

L'événement courant est en RAM. Transmettre l'objet à P2 ne signifie pas nécessairement le déplacer ou le copier. P3 met à jour ses résultats puis le dictionnaire peut être libéré quand il n'est plus référencé. À la fin, P3 retourne le bilan pour affichage dans main.py.

12. Gestion des erreurs

- Ligne vide : ignorer silencieusement.
- JSON malformé : signaler, retourner None et continuer.
- Champ obligatoire absent : signaler, retourner None et continuer.
- Type incorrect : signaler, retourner None et continuer.
- Date non convertible : signaler, retourner None et continuer.

- Fichier vide : cas normal avec zéro événement.
- Fichier introuvable : indiquer le chemin et arrêter la commande.
- Accès refusé ou encodage UTF-8 invalide : signaler et arrêter la commande.

Un chemin inexistant n'est pas forcément "corrompu" : il peut être mal saisi ou le fichier peut avoir été déplacé ou supprimé.

try tente une opération susceptible d'échouer ; except prévoit une réaction à une erreur précise. Cette notion n'apparaît pas dans l'inventaire de cours fourni, mais l'énoncé exige une gestion correcte des exceptions : elle sera expliquée avant utilisation. yield ne gère pas les erreurs.

13. Générer les données de test

Distinction documentaire : le professeur impose un générateur lecteur utilisant yield et yield from. Le script qui fabrique des données synthétiques et le minimum de 10 Go sont des ajouts de l'équipe. Les deux restent dans le périmètre P1 convenu.

Un petit fichier permet de tester vite des cas connus : événements valides, ligne vide, JSON cassé, champs manquants, types incorrects et date invalide. Les résultats attendus sont connus à l'avance. Une ligne valide, une vide et une malformée produisent un seul dictionnaire valide.

Le gros fichier sert à vérifier le fonctionnement progressif et la mémoire. Le générateur produit, sérialise et écrit un événement à la fois. Il ne fabrique pas une liste de toutes les lignes.

L'arrêt dépend de la taille réellement écrite : au moins 10 000 000 000 octets UTF-8. On termine la dernière ligne JSON, quitte à dépasser légèrement la cible. La lecture, elle, continue jusqu'à la fin de tous les fichiers fournis, quelle que soit leur taille.

Les données doivent être variées et reproductibles. Le fichier volumineux reste dans data/generated/, ignoré par Git. Partager le script, la commande et les mesures, pas le fichier lui-même.

14. Tests et affichage choisi

Décision pédagogique pour les petits tests : afficher le résultat de chaque ligne, puis un résumé. Ce n'est pas une obligation du professeur.

```
Ligne 1 : événement valide
Ligne 2 : ligne vide ignorée
Ligne 3 : rejetée - JSON malformé
Ligne 4 : rejetée - timestamp manquant
Ligne 5 : événement valide
```

```
Lecture terminée
2 événements valides
2 lignes rejetées
1 ligne vide ignorée
```

Ne pas utiliser un affichage détaillé pour 10 Go : cela ralentirait le traitement.
Garder un bilan final.

Un indice commence souvent à zéro ; le numéro affiché d'une ligne commencera ici à 1. Un compteur indique le nombre de résultats rencontrés : départ 0, première ligne valide 1,

deuxième ligne valide 2.

On garde trois compteurs : événements valides, rejets et lignes vides. Une ligne valide puis une invalide donnent 1 valide et 1 rejet. enumerate(..., start=1) fournit le numéro et la ligne pendant le parcours.

Vérifications prévues : ordre de lecture, fichiers vides, fermeture, multisource, dictionnaire valide, champs/types/conversions, retour None, poursuite après rejet, compteurs attendus, taille générée, dernière ligne complète et absence d'accumulation des logs.

Compter les résultats ne suffit pas : vérifier aussi leur contenu et leurs types, notamment que connection devient un tuple.

15. Organisation du livrable P1

- logengine/ingestion.py : lecture simple et multisource.
- logengine/parsing.py : décodage, contrôles et conversions.
- main.py : arguments, raccordement et affichage.
- scripts/generate_logs.py : génération progressive.
- Tests P1 : validation des comportements attendus.
- README et bilan P1 : commandes, contrat, explications et mesures.

Ne pas développer les modules des autres membres. Le raccordement final dépend de leurs interfaces livrées, notamment les formats des alertes et du bilan P3. Les fichiers déjà créés restent la base à relire et à ajuster après notre analyse.

16. Référentiel des notions et suite

- def et paramètres : fonctions réutilisables.
- str : ligne de texte.
- dict : champs d'un événement.
- for : parcours des lignes et des fichiers.
- yield : production progressive et suspension.
- yield from : délégation à un lecteur.
- return et None : résultat du parsing d'une ligne.
- in : présence des clés.
- isinstance() : contrôle des types.
- len() : nombre d'éléments.
- tuple() : métadonnées immuables.
- or : accepter plusieurs types possibles.
- Compteurs : résumé sans historique des événements.
- enumerate() : numérotation des lignes.
- Modules et imports : séparation des responsabilités.

yield et yield from figurent bien dans l'inventaire fourni du cours sur les fonctions. Les outils de lecture, JSON, dates, arguments, tests et exceptions devront être justifiés et expliqués au moment de leur utilisation. Ne pas ajouter une technique sans besoin identifié.

Ordre de codage : lecteur simple, multisource, parsing, validation et erreurs, affichage des petits tests, générateur, vérifications et documentation. Utiliser le Python de .venv et la bibliothèque standard ; ne pas installer de dépendances inutilement.